

Checking your Docker system

Get information about Docker

- `rpm -q pkg` or `dpkg-query --show pkg`
- `systemctl status` or `service docker.io status`
- `docker info` *System-wide Docker information*
- `docker version` *Docker version information*
- `docker top` *Show info on docker processes*
- `docker diff` *Show container filesystem changes*
- `docker history` *Show container build history*

Removing Containers and Images

Containers and images continuously consume storage. To prevent that:

- Prevent containers during build (`docker build -rm`)
- Don't save a container after a run (`docker run -rm`)
- Remove containers you no longer need (`docker rm`)
- Remove images you no longer need (`docker rmi`)
- Run a script to remove all containers or images at once

Choose what Goes into a Dockerfile File

These instructions can go in your Dockerfile file:

- **FROM:** Identifies the base image (must be first instruction)
- **MAINTAINER:** Identifies author in Author field for the image
- **RUN:** Executes a command while building an image
- **CMD:** Sets command to execute when container starts
- **EXPOSE:** Exposes container ports to host at runtime
- **ENV:** Sets environment variable to pass to runtime command
- **ADD:** Copies files into the container.

Choose what Goes into a Dockerfile File (cont.)

- **ENTRYPOINT:** Sets container to run as chosen executable
- **VOLUME:** Mounts storage from host into the container
- **USER:** Identifies the user assigned to running the container
- **WORKDIR:** Sets current working directory for RUN, CMD and ENTRYPOINT commands run in the container
- **ONBUILD:** Used when building base images, to add instructions a subsequent build would use to add code or data to the image that were not available during the initial base build

General Rules for Building Containers

- Use root privilege to build containers, non-root user to run
- Choose standard base images
- Pull specific tag when pulling an image to:
 - Use less space
 - Identify exact image used
- Use ENV instructions to store useful information
- Restrict container to one process (no extra access like ssh)
- Don't use / as your build directory

Using Software Repositories

- Use tested, well-know repositories
- Use software packaged in standard formats (deb, rpm, etc.)
- Clean up install cache to keep containers lean
- If available, use keys to check packages you install
- To add repositories to install within a container:
 - **Fedora:** add .repo file /etc/yum.repos.d
 - **Ubuntu:** add .list file to /etc/apt/sources.list.d/
 - **RHEL:** Containers get subscriptions from build host

Accessing Networking from Containers

- Docker starts a private network (such as 172.17.42.0/16)
- The docker0 network routes information:
 - Between containers
 - To other network interfaces on host (set to packet forward)
- Containers are assigned IP addresses on that network
- Containers in the same pod can connect via local host
- Docker networking initiatives are working to make network configurations that span multiple hosts

Base Images

- Small copy of an operating system in a container
- Basic tools and software tools (yum, apt-get)
- Provided by Linux distros (standard and bare bones)
- Examples include official container repositories:
 - **Ubuntu:** https://hub.docker.com/_/ubuntu/
 - **Fedora:** https://hub.docker.com/_/fedora/
 - **CentOS:** https://hub.docker.com/_/centos/
 - **Red Hat Enterprise Linux:**
<https://access.redhat.com/containers#/category/Base%20Image>

Builder Images

- Include operating system plus runtime components
- Developers can add their own source code
- Builder images for different versions of:
 - Python
 - Node.js
 - JBoss Realtime Decision Server, Intelligent Process Server, Web Server (Tomcat),
 - Ruby
 - Apache
- Source-to-image (s2i) injects source code in containers

Custom Base Images

- Allows complete control of image contents
- Has no parent container image
- Created from a software tarball or by just adding executable
- Either FROM scratch or no FROM line
- Instructions for creating custom base images:

<https://docs.docker.com/engine/userguide/eng-image/baseimages/>

- Example: Creating a CentOS base image:

<https://github.com/moby/moby/blob/master/contrib/mkimage-yum.sh>